**LAPORAN PRAKTIKUM**

**STRUKTUR DATA**

**SINGLE LINKED LIST**


**Disusun Oleh:**

Gina Ramadhani

Nim: 2511533014

Dosen Pengampu: DR. Wahyudi, S.T, M.T

Asisten Praktikum: Sherly Sukmadira

**DAPERTEMEN INFORMATIKA**

**FAKULTAS TEKNOLOGI INFORMASI**

**UNIVERSITAS ANDALAS**


**TAHUN 2026**

# KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat, karunia, dan hidayah-Nya, penulis dapat menyelesaikan laporan praktikum Struktur Data dengan judul *Single Linked List* ini dengan baik dan tepat waktu. Laporan ini disusun sebagai bentuk pertanggungjawaban atas kegiatan praktikum yang telah dilaksanakan, sekaligus sebagai sarana untuk memperdalam pemahaman mengenai konsep struktur data dinamis, khususnya *Single Linked List*. Dalam proses penyusunan laporan ini, penulis berusaha menyajikan materi secara sistematis dan jelas agar mudah dipahami oleh pembaca.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi penulisan maupun penyajian materi. Hal ini tidak terlepas dari keterbatasan pengetahuan dan pengalaman yang dimiliki oleh penulis. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari pembaca demi perbaikan di masa yang akan datang. Akhir kata, penulis berharap laporan ini dapat memberikan manfaat, menambah wawasan, serta menjadi referensi bagi pembaca dalam memahami konsep dan penerapan *Single Linked List* dalam pemrograman.

Padang, 2026

Penulis

## DAFTAR ISI

**KATA PENGANTAR** ..... **i**
**DAFTAR ISI** ..... **ii**
**BAB I PENDAHULUAN** ..... **1**
1.1 Latar Belakang ..... 1
1.2 Tujuan Praktikum ..... 1
1.3 Manfaat Praktikum ..... 1
**BAB II PEMBAHASAN** ..... **2**
2.1 Dasar Teori ..... 2
2.2 Praktikum Class “NodeSLL_2511533014” ..... 3
2.3 Praktikum Class “TambahSLL_2511533014” ..... 5
2.4 Praktikum Class “PencarianSLL_2511533014” ..... 14
2.5 Praktikum Class “HapusSLL_2511533014” ..... 18
2.6 Laporan Tugas Praktikum ..... 27
**BAB III PENUTUP** ..... **45**
3.1 Kesimpulan ..... 45
**DAFTAR PUSTAKA** ..... **46**

# BAB I
# PENDAHULUAN

## 1.1 Latar Belakang

Dalam pemrograman, struktur data memiliki peran penting dalam pengelolaan data secara efisien. Salah satu struktur data yang sering digunakan adalah *Linked List*, khususnya *Single Linked List*, yang memungkinkan penyimpanan data secara dinamis. Berbeda dengan array yang menggunakan alokasi memori statis, *Single Linked List* menggunakan pointer untuk menghubungkan setiap elemen. Oleh karena itu, pemahaman tentang konsep ini sangat penting untuk meningkatkan efisiensi dalam pengolahan data, terutama pada kasus yang membutuhkan fleksibilitas tinggi.

## 1.2 Tujuan Praktikum

1. Memahami konsep dasar *Single Linked List*.
2. Mengetahui cara kerja node dan pointer.
3. Mampu mengimplementasikan operasi dasar seperti insert, delete, dan traversal.
4. Melatih kemampuan dalam mengelola struktur data dinamis.

## 1.3 Manfaat Praktikum

1. Menambah pemahaman tentang struktur data dinamis.
2. Meningkatkan kemampuan pemrograman, khususnya dalam penggunaan pointer.
3. Membantu dalam menyelesaikan permasalahan yang membutuhkan struktur data fleksibel.
4. Menjadi dasar untuk mempelajari struktur data yang lebih kompleks.

# BAB II
# PEMBAHASAN

## 2.1 Dasar Teori

*Single Linked List* adalah struktur data yang terdiri dari kumpulan node yang saling terhubung melalui pointer. Setiap node menyimpan data dan alamat node berikutnya. Struktur ini bersifat dinamis karena ukuran memori dapat berubah sesuai kebutuhan selama program berjalan.

Komponen dan Konsep yang Digunakan dalam Single Linked List:

1. ***Node***

   Elemen dasar dalam linked list yang terdiri dari data dan pointer (next).

2. ***Pointer*/Referensi**

   Digunakan untuk menunjuk ke node berikutnya dalam list.

3. ***Head***

   Node pertama dalam linked list yang menjadi titik awal akses.

4. ***Tail***

   Node terakhir yang biasanya menunjuk ke nilai null.

5. **Operasi *Insert* (Penyisipan)**

   Menambahkan node baru di awal, tengah, atau akhir list.

6. **Operasi *Delete* (Penghapusan)**

   Menghapus node tertentu dari list.

7. ***Traversal***

   Proses menelusuri seluruh node dari head hingga akhir.

8. ***Dynamic Memory Allocation***

   Pengalokasian memori secara dinamis saat program berjalan.

## 2.2 Praktikum Class "NodeSLL_2511533014"

```java
package pekan5_2511533014;

public class NodeSLL_2511533014 {
    // node bagian data
    int data_3014;
    // Pointer ke node berikutnya
    NodeSLL_2511533014 next_3014;
    // Konstruktor menginisialisasi node dengan data
    public NodeSLL_2511533014 (int data_3014)
    {
        this.data_3014 = data_3014;
        this.next_3014 = null;
    } }
```

**Gambar 2.1 Kode program praktikum class NodeSLL**

Kode program tersebut merupakan implementasi kelas Node pada struktur data *Single Linked List* dalam bahasa Java. Kelas ini digunakan untuk merepresentasikan satu elemen (node) yang terdiri dari dua bagian utama, yaitu data yang disimpan dan referensi (pointer) ke node berikutnya. Konstruktor pada kelas ini berfungsi untuk menginisialisasi nilai data saat node dibuat serta mengatur pointer next bernilai null, yang menandakan bahwa node tersebut belum terhubung ke node lain.

Berikut penjelasan langkah-langkah kerjanya:

1. ***package pekan5_2511533014;***

   Menyatakan bahwa kelas ini berada dalam package bernama *pekan5_2511533014*.

2. ***public class NodeSLL_2511533014 {***

   Mendefinisikan kelas publik bernama *NodeSLL_2511533014* yang digunakan sebagai node dalam linked list.

3. ***int data_3014;***

   Mendeklarasikan variabel bertipe integer untuk menyimpan data pada node.

4. ***NodeSLL_2511533014 next_3014;***

   Mendeklarasikan variabel referensi (pointer) yang menunjuk ke node berikutnya dalam linked list.

5. ***public NodeSLL_2511533014 (int data_3014)***

   Merupakan konstruktor kelas yang dipanggil saat objek node dibuat dengan parameter nilai data.

6. **{**

   Awal blok konstruktor.

7. ***this.data_3014 = data_3014;***

   Menginisialisasi atribut data_3014 dengan nilai yang diberikan saat pembuatan objek.

8. **this.next_3014 = null;**

   Mengatur pointer next_3014 menjadi null, menandakan belum ada node berikutnya.

9. **}**

   Penutup blok konstruktor.

10. **}**

    Penutup kelas NodeSLL_2511533014.

## 2.3 Praktikum Class “TambahSLL_2511533014”

```java
package pekan5_2511533014;

public class TambahSLL_2511533014 {
    public static NodeSLL_2511533014 insertAtFront (NodeSLL_2511533014 head, int value) {
        NodeSLL_2511533014 new_node = new NodeSLL_2511533014 (value);
        new_node.next_3014 = head;
        return new_node;
    }
    // fungsi menambahkan node di akhir SLL
    public static NodeSLL_2511533014 insertAtEnd (NodeSLL_2511533014 head, int value) {
        // buat sebuah node dengan sebuah nilai
        NodeSLL_2511533014 newNode = new NodeSLL_2511533014 (value);
        // jika list kosong maka node jadi head
        if (head == null) {
            return newNode;
        }
        // Simpan head ke variabel sementara
        NodeSLL_2511533014 last = head;
        // telusuri ke node akhir
        while (last.next_3014 != null) {
            last = last.next_3014;
        }
        // ubah pointer
        last.next_3014 = newNode;
        return head;
    }
    static NodeSLL_2511533014 GetNode (int data_3014) {
        return new NodeSLL_2511533014 (data_3014);
    }

    static NodeSLL_2511533014 insertPos (NodeSLL_2511533014 headNode, int position, int value) {
        NodeSLL_2511533014 head = headNode;
        if (position < 1)
            System.out.print ("Invalid position");
        if (position == 1) {
            NodeSLL_2511533014 new_node = new NodeSLL_2511533014(value);
            new_node.next_3014 = head;
            return new_node;
        } else {
            while (position-- != 0) {
                if (position == 1) {
                    NodeSLL_2511533014 newNode = GetNode (value);
                    newNode.next_3014 = headNode.next_3014;
                    headNode.next_3014 = newNode;
                    break;
                }
                headNode = headNode.next_3014;
            }
            if (position != 1)
        System.out.print("Posisi di luar jangkauan");
        return head;    }  }
    public static void printList_3014 (NodeSLL_2511533014 head) {
        NodeSLL_2511533014 curr = head;
     while (curr.next_3014 != null) {
         System.out.print(curr.data_3014+"-->");
         curr = curr.next_3014;
     }
     if (curr.next_3014==null) {
         System.out.print(curr.data_3014);  }
     System.out.println();
     }
```

**Gambar 2.2 Kode program praktikum class TambahSLL**

```java
    public static void main (String[] args) {
        // buat linked list 2->3->5->6
        NodeSLL_2511533014 head = new NodeSLL_2511533014(2);
        head.next_3014 = new NodeSLL_2511533014(3);
        head.next_3014.next_3014 = new NodeSLL_2511533014(5);
        head.next_3014.next_3014.next_3014 = new NodeSLL_2511533014(6);
        // cetak list asli
        System.out.print("Senarai berantai awal:");
        printList_3014 (head);
        // tambahkan node baru didepan
        System.out.print("tambah 1 simpul di depan: ");
        int data_3014 = 1;
        head = insertAtFront(head, data_3014);
        // cetak update list
        printList_3014(head);
        // tambahkan node baru dibelakang
        System.out.print("tambah 1 simpul di belakang");
        int data2 = 7;
        head = insertAtEnd (head, data2);
        // cetak update list
        printList_3014 (head);
        System.out.print("tambah 1 simpul ke data 4: ");
        int data3 = 4;
        int pos=4;
        head = insertPos (head,pos,data3);
        // cetak update list
        printList_3014 (head);

    }
        }
```

**Gambar 2.3 Sambungan Kode program praktikum class TambahSLL**

Kode program tersebut merupakan implementasi operasi dasar pada struktur data *Single Linked List* dalam bahasa Java, khususnya untuk menambahkan node di berbagai posisi, yaitu di depan (*insert at front*), di akhir (*insert at end*), dan pada posisi tertentu (*insert at position*), serta menampilkan isi linked list. Program ini juga memiliki fungsi utama (*main*) yang digunakan untuk membuat linked list awal, kemudian melakukan beberapa operasi penambahan node, dan menampilkan hasilnya untuk melihat perubahan yang terjadi setelah setiap operasi dilakukan.

Berikut Penjelasan langkah-langkah kerjanya:

1. package pekan5_2511533014;

   Menyatakan bahwa kelas berada dalam package *pekan5_2511533014*.

2. public class TambahSLL_2511533014 {

Mendefinisikan kelas utama untuk operasi penambahan pada *Single Linked List*.

**Fungsi insertAtFront**

3. public static NodeSLL_2511533014 insertAtFront (NodeSLL_2511533014 head, int value) {

    - Membuat method untuk menambahkan node di bagian depan linked list.
    - Parameter:

        1. head = node pertama linked list
        2. value = data yang ingin ditambahkan

4. NodeSLL_2511533014 new_node = new NodeSLL_2511533014 (value);

    Membuat node baru dengan nilai yang diberikan.

5. new_node.next_3014 = head;

    Node baru menunjuk ke head lama.

6. return new_node;

    Node baru menjadi head yang baru.

7. }

    Pentup method insertAtFront

8. NodeSLL_2511533014 newNode = new NodeSLL_2511533014 (value);

    Membuat node baru dengan nilai tertentu.

9. if (head == null) {

    Mengecek apakah linked list kosong.

10. return newNode;

    Jika kosong maka node baru langsung menjadi head.

11. }

    Penutup blok if.

12. NodeSLL_2511533014 last = head;

    Variabel last digunakan untuk menelusuri list.

13. while (last.next_3014 != null) {

    last = last.next_3014;

    }

    - Perulangan mencari node terakhir.
    - Selama next masih ada, pindah ke node berikutnya.

14. last.next_3014 = newNode;

    Node terakhir dihubungkan ke node baru.

15. return head;

    Mengembalikan head awal.

    **Method GetNode**

16. static NodeSLL_2511533014 GetNode (int data_3014) {

    Method pembuat node.

17. return new NodeSLL_2511533014 (data_3014);

    Mengembalikan node baru berisi data.

**Method insertPos**

18. static NodeSLL_2511533014 insertPos (NodeSLL_2511533014 headNode, int position, int value) {

    Method untuk menambah node pada posisi tertentu.

19. NodeSLL_2511533014 head = headNode;

    Menyimpan head asli.

20. if (position < 1)

        System.out.print ("Invalid position");

    Jika posisi kurang dari 1 maka posisi tidak valid.

21. if (position == 1) {

    Jika ingin menambah di posisi pertama.

22. NodeSLL_2511533014 new_node = new NodeSLL_2511533014(value);

    Membuat node baru.

23. new_node.next_3014 = head;

    Node baru menunjuk ke head lama.

24. return new_node;

    Node baru menjadi head.

25. else {

    Jika posisi bukan 1.

26. while (position-- != 0) {

Perulangan menuju posisi yang diinginkan.

27. if (position == 1) {

Saat sudah sampai sebelum posisi tujuan.

28. NodeSLL_2511533014 newNode = GetNode (value);

Membuat node baru.

29. newNode.next_3014 = headNode.next_3014;

Node baru menunjuk ke node berikutnya.

30. headNode.next_3014 = newNode;

Node sebelumnya dihubungkan ke node baru.

31. break;

Menghentikan  perulangan

32. headNode = headNode.next_3014;

Pindah ke node berikutnya

33. if (position != 1)

System.out.print("Posisi di luar jangkauan");

Jika posisi melebihi panjang list maka muncul pesan error.

34. return head;

Mengembalikan head awal

**Method printList**

35. public static void printList_3014 (NodeSLL_2511533014 head) {

Method untuk mencetak isi linked list.

36. NodeSLL_2511533014 curr = head;

Variabel bantu untuk traversal.

37. while (curr.next_3014 != null) {

Selama belum node terakhir.

38. System.out.print(curr.data_3014+"-->");

Cetak data node

39. curr = curr.next_3014;

Pindah ke node berikutnya.

40. if (curr.next_3014==null) {

Jika sudah node terakhir.

41. System.out.print(curr.data_3014);

Cetak node terakhir tanpa tanda panah.

42. System.out.println();

Pindah baris

**Method main**

43. public static void main (String[] args) {

Program utama dijalankan dari sini.

44. NodeSLL_2511533014 head = new NodeSLL_2511533014(2);

Membuat head bernilai 2

**Membuat Linked List Awal**

45. NodeSLL_2511533014 head = new NodeSLL_2511533014(2);

    Membuat head bernilai 2.

46. head.next_3014 = new NodeSLL_2511533014(3);

    Node berikutnya bernilai 3.

47. head.next_3014.next_3014 = new NodeSLL_2511533014(5);

    Node berikutnya bernilai 5.

48. head.next_3014.next_3014.next_3014 = new NodeSLL_2511533014(6);

    Node berikutnya bernilai 6.

**Cetak List Awal**

49. System.out.print("Senarai berantai awal:");

    Menampilkan teks

50. printList_3014 (head);

    Mencetak linked list

**Tambah Node di Depan**

51. int data_3014 = 1;

    Data yang akan ditambah

52. head = insertAtFront(head, data_3014);

    Menambah node 1 di depan.

**Tambah Node di Belakang**

53. int data2 = 7;

    Data baru = 7

54. head = insertAtEnd (head, data2);

    Menambah node di belakang

    **Tambah Node di Posisi Tertentu**

55. int data3 = 4;

    int pos=4;

    Tambah angka 4 di posisi ke-4.

56. head = insertPos (head,pos,data3);

    Menjalankan insert posisi

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.4 dibawah ini.

```
<terminated> TambahSLL_2511533014 [Java Application] C:\Users\rara0\.p
Senarai berantai awal:2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7
```

**Gambar 2.4 Output kode program praktikum class TambahSLL_2511533014**

### 2.3.1 Analisis

Output program menunjukkan proses penambahan node pada struktur data Single Linked List (SLL). Awalnya linked list berisi data 2-->3-->5-->6, kemudian program menambahkan node bernilai 1 di bagian depan sehingga menjadi 1-->2-->3--

>5-->6. Setelah itu ditambahkan node 7 di bagian belakang menjadi 1-->2-->3-->5-->6-->7, lalu node 4 disisipkan pada posisi tertentu sehingga hasil akhir linked list menjadi 1-->2-->3-->4-->5-->6-->7. Output ini membuktikan bahwa fungsi penambahan node di depan, belakang, dan tengah linked list berjalan dengan benar.

## 2.4 Praktikum Class "PencarianSLL_2511533014"

```java
package pekan5_2511533014;

public class PencarianSLL_2511533014 {
    static boolean searchKey_3014 (NodeSLL_2511533014 head, int key) {
        NodeSLL_2511533014 curr = head;
        while (curr != null) {
            if (curr.data_3014 == key)
                return true;
            curr = curr.next_3014; }
        return false; }
    public static void traversal_3014 (NodeSLL_2511533014 head) {
        //mulai dari head
        NodeSLL_2511533014 curr = head;
        // telusuri sampai pointer null
        while (curr != null)  {
            System.out.print(" " + curr.data_3014);
            curr = curr.next_3014; }
        System.out.println();      }
        public static void main (String[] args) {
            NodeSLL_2511533014 head = new NodeSLL_2511533014(14);
            head.next_3014 = new NodeSLL_2511533014(21);
            head.next_3014.next_3014 = new NodeSLL_2511533014(13);
            head.next_3014.next_3014.next_3014 = new NodeSLL_2511533014(30);
            head.next_3014.next_3014.next_3014.next_3014 = new NodeSLL_2511533014(10);
            System.out.print ("Penelusuran SLL : ") ;
            traversal_3014 (head);
            // data yang akan dicari
            int key = 30;
            System.out.print("cari data " + key+ "  =  ");
            if (searchKey_3014(head, key))
                System.out.println("ketemu");
            else
                System.out.println("tidak ada");
        }
    }
```

**Gambar 2.5 Kode program praktikum class PencarianSLL**

Program PencarianSLL_2511533014 digunakan untuk melakukan pencarian data pada struktur data Single Linked List (SLL). Program terlebih dahulu membuat linked list berisi data 14, 21, 13, 30, 10, kemudian menampilkan seluruh isi list menggunakan proses traversal. Setelah itu program mencari data tertentu yaitu 30 dengan metode sequential search, yaitu menelusuri node satu per satu dari head hingga data ditemukan atau mencapai

akhir list. Jika data ditemukan maka output menampilkan “ketemu”, sedangkan jika tidak ditemukan akan menampilkan “tidak ada”.

Berikut penjelasan langkah-langkah kerjanya:

1. package pekan5_2511533014;

   → Menentukan package/folder program.

2. public class PencarianSLL_2511533014 {

   → Membuat class bernama PencarianSLL_2511533014.

3. static boolean searchKey_3014 (NodeSLL_2511533014 head, int key) {

   → Method untuk mencari data pada linked list dan menghasilkan nilai true atau false.

4. NodeSLL_2511533014 curr = head;

   → Variabel curr digunakan untuk memulai pencarian dari head.

5. while (curr != null) {

   → Perulangan berjalan selama node masih ada.

6. if (curr.data_3014 == key)

   → Mengecek apakah data pada node sama dengan data yang dicari.

7. return true;

   → Jika data ditemukan maka mengembalikan nilai true.

8. curr = curr.next_3014;

   → Pindah ke node berikutnya.

9. return false;

→ Jika data tidak ditemukan sampai akhir list maka mengembalikan false.

10. public static void traversal_3014 (NodeSLL_2511533014 head) {

→ Method untuk menampilkan isi linked list.

11. NodeSLL_2511533014 curr = head;

→ Variabel curr mulai dari head.

12. while (curr != null) {

→ Menelusuri seluruh node sampai akhir.

13. System.out.print(" " + curr.data_3014);

→ Menampilkan data tiap node.

14. curr = curr.next_3014;

→ Berpindah ke node selanjutnya.

15. System.out.println();

→ Pindah ke baris baru.

16. public static void main (String[] args) {

→ Method utama program.

17. NodeSLL_2511533014 head = new NodeSLL_2511533014(14);

→ Membuat node pertama (head) berisi data 14.

18. head.next_3014 = new NodeSLL_2511533014(21);

→ Menambahkan node kedua berisi 21.

19. head.next_3014.next_3014 = new NodeSLL_2511533014(13);

→ Menambahkan node ketiga berisi 13.

20. head.next_3014.next_3014.next_3014 = new NodeSLL_2511533014(30);

→ Menambahkan node keempat berisi 30.

21. head.next_3014.next_3014.next_3014.next_3014 = new NodeSLL_2511533014(10);

→ Menambahkan node kelima berisi 10.

22. System.out.print ("Penelusuran SLL : ");

→ Menampilkan teks “Penelusuran SLL”.

23. traversal_3014 (head);

→ Memanggil method traversal untuk mencetak isi linked list.

24. int key = 30;

→ Menentukan data yang akan dicari yaitu 30.

25. System.out.print("cari data " + key+ "  =  ");

→ Menampilkan teks pencarian data.

26. if (searchKey_3014(head, key))

→ Mengecek apakah data ditemukan di linked list.

27. System.out.println("ketemu");

→ Ditampilkan jika data ditemukan.

28. else

→ Kondisi jika data tidak ditemukan.

29. System.out.println("tidak ada");

→ Ditampilkan jika data tidak ada dalam linked list.

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.6 dibawah ini.

```
<terminated> PencarianSLL_2511533014 [Java App
Penelusuran SLL :  14 21 13 30 10
cari data 30  =  ketemu
```

**Gambar 2.6 Output kode program praktikum class PencarianSLL_2511533014**

### 2.4.1 Analisis

Output program menunjukkan bahwa proses traversal dan pencarian pada Single Linked List (SLL) berjalan dengan benar. Baris pertama menampilkan isi linked list yaitu 14 21 13 30 10 hasil dari method traversal_3014() yang menelusuri node dari head hingga node terakhir. Selanjutnya program melakukan pencarian data 30 menggunakan method searchKey_3014(). Karena nilai 30 ditemukan pada salah satu node linked list, maka output menampilkan tulisan ketemu. Ini membuktikan bahwa algoritma pencarian sequential pada linked list berhasil dijalankan dengan baik.

## 2.5 Praktikum Class "HapusSLL_2511533014"

```java
package pekan5_2511533014;

public class HapusSLL_2511533014 {
    // fungsi untuk menghapus head
    public static NodeSLL_2511533014 deleteHead_3014(NodeSLL_2511533014 head_3014) {
        // jika SLL kosong
        if (head_3014 == null) {
            return null;
        }
        // pindahkan head ke node berikutnya
        head_3014 = head_3014.next_3014;
        // return head baru
        return head_3014;
    }
    // fungsi menghapus node terakhir SLL
    public static NodeSLL_2511533014 removeLastNode_3014(NodeSLL_2511533014 head_3014) {
        // jika list kosong , return null
        if (head_3014 == null) {
            return null;
        }
        // jika list satu node , hapus node dan return null
        if (head_3014.next_3014 == null) {
            return null;
        }
        // temukan node terakhir kedua
        NodeSLL_2511533014 secondLast = head_3014;
        while (secondLast.next_3014.next_3014 != null) {
            secondLast = secondLast.next_3014;
        }
        // hapus node terakhir
        secondLast.next_3014 = null;
        return head_3014;        }
```

```java
    // fungsi menghapus node di posisi tertentu
    public static NodeSLL_2511533014 deleteNode_3014(NodeSLL_2511533014 head_3014, int position_3014) {
        NodeSLL_2511533014 temp_3014 = head_3014;
        NodeSLL_2511533014 prev_3014 = null;
        // jika linked list null
        if (temp_3014 == null)
            return head_3014;
        // kasus 1: head dihapus
        if (position_3014 == 1) {
            head_3014 = temp_3014.next_3014;
            return head_3014;
        }
        // kasus 2: menghapus node di tengah
        for (int i_3014 = 1; temp_3014 != null && i_3014 < position_3014; i_3014++) {
            prev_3014 = temp_3014;
            temp_3014 = temp_3014.next_3014;
        }
        // jika ditemukan, hapus node
        if (temp_3014 != null) {
            prev_3014.next_3014 = temp_3014.next_3014;
        } else {
            System.out.println("Data tidak ada");
        }
        return head_3014;
    }
    // fungsi mencetak SLL
    public static void printList_3014(NodeSLL_2511533014 head_3014) {
        NodeSLL_2511533014 curr_3014 = head_3014;
        while (curr_3014.next_3014 != null) {
            System.out.print(curr_3014.data_3014 + "-->");
            curr_3014 = curr_3014.next_3014;
        }
        System.out.print(curr_3014.data_3014);
        System.out.println();
    }
```

```java
    // kelas main
    public static void main(String[] args) {
        // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
        NodeSLL_2511533014 head_3014 = new NodeSLL_2511533014(1);
        head_3014.next_3014 = new NodeSLL_2511533014(2);
        head_3014.next_3014.next_3014 = new NodeSLL_2511533014(3);
        head_3014.next_3014.next_3014.next_3014 = new NodeSLL_2511533014(4);
        head_3014.next_3014.next_3014.next_3014.next_3014 = new NodeSLL_2511533014(5);
        head_3014.next_3014.next_3014.next_3014.next_3014.next_3014 = new NodeSLL_2511533014(6);
        // cetak list awal
        System.out.println("List awal: ");
        printList_3014(head_3014);
        // hapus head
        head_3014 = deleteHead_3014(head_3014);
        System.out.println("List setelah head dihapus: ");
        printList_3014(head_3014);
        // hapus node terakhir
        head_3014 = removeLastNode_3014(head_3014);
        System.out.println("List setelah simpul terakhir dihapus: ");
        printList_3014(head_3014);
        // hapus node di posisi 2
        int position_3014 = 2;
        head_3014 = deleteNode_3014(head_3014, position_3014);
        System.out.println("List setelah posisi 2 dihapus: ");
        printList_3014(head_3014);
    } }
```

**Gambar 2.7 Kode Program praktikum class HapusSLL**

Program HapusSLL_2511533014 digunakan untuk menghapus node pada struktur data Single Linked List (SLL). Program memiliki tiga operasi utama yaitu menghapus node pertama (head), menghapus node terakhir, dan menghapus node pada posisi tertentu. Linked list awal berisi data 1-->2-->3-->4-->5-->6, kemudian program menghapus head sehingga node 1 hilang,

menghapus node terakhir sehingga 6 hilang, lalu menghapus node pada posisi ke-2 sehingga data pada posisi tersebut ikut terhapus. Program juga menampilkan isi linked list setiap selesai proses penghapusan untuk menunjukkan perubahan data.

Berikut penjelasan langkah-langkah kerjanya:

1. package pekan5_2511533014;

   → Menentukan package program.

2. public class HapusSLL_2511533014 {

   → Membuat class bernama HapusSLL_2511533014.

   **Method deleteHead**

3. public static NodeSLL_2511533014 deleteHead_3014(NodeSLL_2511533014 head_3014) {

   → Method untuk menghapus node pertama (head).

4. if (head_3014 == null) {

   return null;

   }

   → Jika linked list kosong maka mengembalikan null.

5. head_3014 = head_3014.next_3014;

   → Head dipindahkan ke node berikutnya.

6. return head_3014;

   → Mengembalikan head baru.

**Method removeLastNode**

7. public static NodeSLL_2511533014 removeLastNode_3014(NodeSLL_2511533014 head_3014) {

   → Method untuk menghapus node terakhir.

8. if (head_3014 == null) {

   return null;

   }

   → Jika list kosong maka return null.

9. if (head_3014.next_3014 == null) {

   return null;

   }

   → Jika hanya ada satu node maka node dihapus dan list menjadi kosong.

10. NodeSLL_2511533014 secondLast = head_3014;

    → Variabel untuk mencari node sebelum terakhir.

11. while (secondLast.next_3014.next_3014 != null) {

    → Menelusuri sampai node sebelum terakhir.

12. secondLast = secondLast.next_3014;

    → Pindah ke node berikutnya.

13. secondLast.next_3014 = null;

    → Menghapus node terakhir dengan membuat pointer bernilai null.

14. return head_3014;

→ Mengembalikan head.

**Method deleteNode**

15. public static NodeSLL_2511533014 deleteNode_3014(NodeSLL_2511533014 head_3014, int position_3014) {

→ Method untuk menghapus node pada posisi tertentu.

16. NodeSLL_2511533014 temp_3014 = head_3014;

→ Variabel sementara untuk traversal.

17. NodeSLL_2511533014 prev_3014 = null;

→ Variabel untuk menyimpan node sebelumnya.

18. if (temp_3014 == null)

    return head_3014;

→ Jika list kosong maka langsung return.

19. if (position_3014 == 1) {

→ Jika node pertama yang dihapus.

20. head_3014 = temp_3014.next_3014;

→ Head dipindahkan ke node berikutnya.

21. return head_3014;

→ Mengembalikan head baru.

22. for (int i_3014 = 1; temp_3014 != null && i_3014 < position_3014; i_3014++) {

→ Perulangan menuju posisi yang akan dihapus.

23. prev_3014 = temp_3014;

→ Menyimpan node sebelumnya.

24. temp_3014 = temp_3014.next_3014;

→ Pindah ke node berikutnya.

25. if (temp_3014 != null) {

→ Jika node ditemukan.

26. prev_3014.next_3014 = temp_3014.next_3014;

→ Node dihapus dengan menyambungkan node sebelumnya ke node sesudahnya.

27. System.out.println("Data tidak ada");

→ Ditampilkan jika posisi tidak ditemukan.

28. return head_3014;

→ Mengembalikan head

**Method printList**

29. public static void printList_3014(NodeSLL_2511533014 head_3014) {

→ Method untuk mencetak isi linked list.

30. NodeSLL_2511533014 curr_3014 = head_3014;

→ Variabel traversal.

31. while (curr_3014.next_3014 != null) {

→ Menelusuri node sampai akhir.

32. System.out.print(curr_3014.data_3014 + "-->");

→ Menampilkan data node.

33. curr_3014 = curr_3014.next_3014;

→ Pindah ke node berikutnya.

34. System.out.print(curr_3014.data_3014);

→ Menampilkan node terakhir.

35. System.out.println();

→ Pindah ke baris baru.

**Method main**

36. public static void main(String[] args) {

→ Method utama program.

37. NodeSLL_2511533014 head_3014 = new NodeSLL_2511533014(1);

→ Membuat head bernilai 1.

38. head_3014.next_3014 = new NodeSLL_2511533014(2);

→ Menambahkan node 2.

39. head_3014.next_3014.next_3014 = new NodeSLL_2511533014(3);

→ Menambahkan node 3.

40. head_3014.next_3014.next_3014.next_3014 = new NodeSLL_2511533014(4);

→ Menambahkan node 4.

41. head_3014.next_3014.next_3014.next_3014.next_3014 = new NodeSLL_2511533014(5);

→ Menambahkan node 5.

42. head_3014.next_3014.next_3014.next_3014.next_3014.next_3014 = new NodeSLL_2511533014(6);

→ Menambahkan node 6.

43. System.out.println("List awal: ");

→ Menampilkan teks list awal.

44. printList_3014(head_3014);

→ Mencetak linked list awal.

45. head_3014 = deleteHead_3014(head_3014);

→ Menghapus node pertama.

46. System.out.println("List setelah head dihapus: ");

→ Menampilkan teks setelah head dihapus.

47. printList_3014(head_3014);

→ Menampilkan hasil list terbaru.

48. head_3014 = removeLastNode_3014(head_3014);

→ Menghapus node terakhir.

49. System.out.println("List setelah simpul terakhir dihapus: ");

→ Menampilkan teks penghapusan node terakhir.

50. printList_3014(head_3014);

→ Menampilkan list terbaru.

51. int position_3014 = 2;

→ Menentukan posisi node yang akan dihapus.

52. head_3014 = deleteNode_3014(head_3014, position_3014);

→ Menghapus node pada posisi ke-2.

53. System.out.println("List setelah posisi 2 dihapus: ");

→ Menampilkan teks hasil penghapusan posisi 2.

54. printList_3014(head_3014);

→ Menampilkan linked list akhir.

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.8 dibawah ini.

```
<terminated> HapusSLL_2511533014 [Java Applicati
List awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir dihapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5
```

**Gambar 2.8 Output kode program praktikum class HapusSLL_2511533014**

### 2.5.1 Analisis

Output program menunjukkan proses penghapusan node pada Single Linked List (SLL) berjalan dengan benar. Awalnya linked list berisi 1-->2-->3-->4-->5-->6, kemudian setelah method deleteHead_3014() dijalankan, node pertama yaitu 1 berhasil dihapus sehingga list menjadi 2-->3-->4-->5-->6. Selanjutnya method removeLastNode_3014() menghapus node terakhir yaitu 6, sehingga tersisa 2-->3-->4-->5. Setelah itu method deleteNode_3014() menghapus node pada posisi ke-2, yaitu data 3, sehingga hasil akhir linked list menjadi 2-->4-->5. Output ini membuktikan bahwa operasi penghapusan di awal, akhir, dan posisi tertentu pada linked list berhasil dilakukan dengan baik.

## 2.6 Laporan Tugas Praktikum

### 2.6.1 Tugas Praktikum class "Pasien_2511533014"

```java
package pekan5_2511533014;

public class Pasien_2511533014 {

    // Atribut pasien
    private String namaPasien_3014;
    private String penyakit_3014;
    private int nomorAntrian_3014;

    // Pointer next
    Pasien_2511533014 next_3014;

    // Constructor
    public Pasien_2511533014(String namaPasien, String penyakit, int nomorAntrian) {
        this.namaPasien_3014 = namaPasien;
        this.penyakit_3014 = penyakit;
        this.nomorAntrian_3014 = nomorAntrian;
        this.next_3014 = null;
    }
    // Getter nama pasien
    public String getNamaPasien_3014() {
        return namaPasien_3014;
    }
    // Getter penyakit
    public String getPenyakit_3014() {
        return penyakit_3014;
    }
```

```java
    // Getter nomor antrian
    public int getNomorAntrian_3014() {
        return nomorAntrian_3014;
    }
    // Getter next
    public Pasien_2511533014 getNext_3014() {
        return next_3014;
    }
    // Setter nama pasien
    public void setNamaPasien_3014(String namaPasien) {
        this.namaPasien_3014 = namaPasien;
    }
    // Setter penyakit
    public void setPenyakit_3014(String penyakit) {
        this.penyakit_3014 = penyakit;
    }
    // Setter nomor antrian
    public void setNomorAntrian_3014(int nomorAntrian) {
        this.nomorAntrian_3014 = nomorAntrian;
    }
    // Setter next
    public void setNext_3014(Pasien_2511533014 next) {
        this.next_3014 = next;
    } }
```

Kelas Pasien_2511533014 merupakan kelas ADT (Abstract Data Type) yang digunakan untuk merepresentasikan data pasien dalam struktur Single Linked List. Pada kelas ini terdapat atribut namaPasien_3014, penyakit_3014, dan nomorAntrian_3014 yang berfungsi menyimpan informasi pasien, sedangkan atribut next_3014 digunakan sebagai pointer untuk menghubungkan node pasien satu dengan node berikutnya. Constructor digunakan untuk menginisialisasi data pasien saat objek dibuat. Selain itu, kelas ini juga memiliki method getter untuk mengambil nilai atribut dan method setter untuk mengubah nilai atribut maupun pointer next. Dengan adanya pointer next_3014, setiap objek pasien dapat saling terhubung sehingga membentuk linked list yang digunakan pada sistem antrian rumah sakit.

Berikut penjelasan langkah-langkah kerjanya:

1. package pekan5_2511533014;

→ Menentukan package program.

2. public class Pasien_2511533014 {

→ Membuat class bernama `Pasien_2511533014`.

3. `private String namaPasien_3014;`

→ Mendeklarasikan variabel untuk menyimpan nama pasien.

4. `private String penyakit_3014;`

→ Mendeklarasikan variabel untuk menyimpan keluhan atau penyakit pasien.

5. `private int nomorAntrian_3014;`

→ Mendeklarasikan variabel untuk menyimpan nomor antrian pasien.

6. `Pasien_2511533014 next_3014;`

→ Membuat pointer `next` yang menunjuk ke node berikutnya.

7. `public Pasien_2511533014(String namaPasien, String penyakit, int nomorAntrian) {`

→ Membuat constructor untuk menginisialisasi data pasien.

8. `this.namaPasien_3014 = namaPasien;`

→ Mengisi atribut nama pasien.

9. `this.penyakit_3014 = penyakit;`

→ Mengisi atribut penyakit pasien.

10. `this.nomorAntrian_3014 = nomorAntrian;`

→ Mengisi atribut nomor antrian pasien.

11. `this.next_3014 = null;`

→ Mengatur pointer next menjadi `null`.

12. `public String getNamaPasien_3014() {`

→ Membuat method getter untuk mengambil nama pasien.

13. `return namaPasien_3014;`

→ Mengembalikan nilai nama pasien.

14. `public String getPenyakit_3014() {`

→ Membuat method getter untuk mengambil penyakit pasien.

15. `return penyakit_3014;`

→ Mengembalikan nilai penyakit pasien.

16. `public int getNomorAntrian_3014() {`

→ Membuat method getter untuk mengambil nomor antrian pasien.

17. `return nomorAntrian_3014;`

→ Mengembalikan nomor antrian pasien.

18. `public Pasien_2511533014 getNext_3014() {`

→ Membuat method getter untuk mengambil node berikutnya.

19. `return next_3014;`

→ Mengembalikan pointer next.

20. `public void setNamaPasien_3014(String namaPasien) {`

→ Membuat method setter untuk mengubah nama pasien.

21. `this.namaPasien_3014 = namaPasien;`

→ Mengubah nilai nama pasien.

22. `public void setPenyakit_3014(String penyakit) {`

→ Membuat method setter untuk mengubah penyakit pasien.

23. `this.penyakit_3014 = penyakit;`

→ Mengubah nilai penyakit pasien.

24. `public void setNomorAntrian_3014(int nomorAntrian) {`

→ Membuat method setter untuk mengubah nomor antrian pasien.

25. `this.nomorAntrian_3014 = nomorAntrian;`

→ Mengubah nomor antrian pasien.

26. `public void setNext_3014(Pasien_2511533014 next) {`

→ Membuat method setter untuk menghubungkan node berikutnya.

27. `this.next_3014 = next;`

→ Mengisi pointer next dengan node tujuan.

28. `}`

→ Menutup class `Pasien_2511533014`.

### 2.6.2 Tugas Praktikum class “RumahSakit_2511533014”

```java
package pekan5_2511533014;
import java.util.Scanner;
public class RumahSakit_2511533014 {
    static Pasien_2511533014 head_3014 = null;
    static int counter_3014 = 0;
    // Insert at tail
    public static void daftarkanPasien_3014(String namaPasien, String penyakit) {
        counter_3014++;
        Pasien_2511533014 pasienBaru_3014 =
                new Pasien_2511533014(namaPasien, penyakit, counter_3014);
        if (head_3014 == null) {

            head_3014 = pasienBaru_3014;

        } else {
            Pasien_2511533014 bantu_3014 = head_3014;
            while (bantu_3014.next_3014 != null)
                bantu_3014 = bantu_3014.next_3014;
            bantu_3014.next_3014 = pasienBaru_3014;
        }
        System.out.println("Pasien berhasil didaftarkan!");
        System.out.println("Nomor Antrian: "
                + pasienBaru_3014.getNomorAntrian_3014());
    }
    // Delete head
    public static void panggilPasien_3014() {
        if (head_3014 == null) {
            System.out.println("Antrian kosong!");
            return;
        }
        System.out.println("\n=== Pasien Dipanggil ===");
        System.out.println("Nomor Antrian : "
                + head_3014.getNomorAntrian_3014());
        System.out.println("Nama Pasien   : "
                + head_3014.getNamaPasien_3014());
        System.out.println("Keluhan       : "
                + head_3014.getPenyakit_3014());

        head_3014 = head_3014.next_3014;
    }
    // Display
    public static void tampilkanAntrian_3014() {
        if (head_3014 == null) {
            System.out.println("Antrian masih kosong!");
            return;
        }
        System.out.println("\n=== Daftar Antrian Pasien ===");
        Pasien_2511533014 bantu_3014 = head_3014;
        int posisi_3014 = 1;
        while (bantu_3014 != null) {
            System.out.println("Posisi Antrian : "
```

```
                        + posisi_3014);
                System.out.println("Nomor Antrian  : "
                        + bantu_3014.getNomorAntrian_3014());
                System.out.println("Nama Pasien    : "
                        + bantu_3014.getNamaPasien_3014());
                System.out.println("Keluhan        : "
                        + bantu_3014.getPenyakit_3014());
                System.out.println("--------------------------");
                bantu_3014 = bantu_3014.next_3014;
                posisi_3014++;
            }
        }
        // Search pasien
        public static void cariPasien_3014(String namaCari_3014) {
            if (head_3014 == null) {
                System.out.println("Antrian masih kosong!");
                return;
            }
            Pasien_2511533014 bantu_3014 = head_3014;
            while (bantu_3014 != null) {
                if (bantu_3014.getNamaPasien_3014()
                        .equalsIgnoreCase(namaCari_3014)) {
                    System.out.println("\n=== Pasien Ditemukan ===");
                    System.out.println("Nomor Antrian : "
                            + bantu_3014.getNomorAntrian_3014());
                    System.out.println("Nama Pasien   : "
                            + bantu_3014.getNamaPasien_3014());
                    System.out.println("Keluhan       : "
                            + bantu_3014.getPenyakit_3014());
                    return;   }
                bantu_3014 = bantu_3014.next_3014;        }
            System.out.println("Pasien tidak ditemukan!");
        }
        // Cek status antrian
        public static void cekStatusAntrian_3014() {
            if (head_3014 == null) {
                System.out.println("Antrian kosong!");
                return;
            }
            Pasien_2511533014 bantu_3014 = head_3014;
            int jumlah_3014 = 0;
            while (bantu_3014 != null) {
                jumlah_3014++;
                bantu_3014 = bantu_3014.next_3014;
            }
            System.out.println("=== Status Antrian ===");
            System.out.println("Jumlah Pasien : "  + jumlah_3014);
            System.out.println("Pasien Terdepan:");
            System.out.println("Nomor Antrian : "  + head_3014.getNomorAntrian_3014());
            System.out.println("Nama Pasien   : "  + head_3014.getNamaPasien_3014());
```

```
104         System.out.println("Keluhan       : "+
    head_3014.getPenyakit_3014());
105     }
106     // Main program
107     public static void main(String[] args) {
108         Scanner input_3014 = new Scanner(System.in);
109         int pilihan_3014;
110         do {
111             System.out.println("\n=== Antrian Rumah Sakit
    NIM: 2511533014 ===");
112             System.out.println("1. Daftarkan Pasien
    (Insert)");
113             System.out.println("2. Panggil Pasien (Delete
    Head)");
114             System.out.println("3. Tampilkan Antrian
    (Display)");
115             System.out.println("4. Cari Pasien (Search)");
116             System.out.println("5. Cek Status Antrian");
117             System.out.println("6. Keluar");
118             System.out.print("Pilihan: ");
119             pilihan_3014 = input_3014.nextInt();
120             input_3014.nextLine();
121             if (pilihan_3014 == 1) {
122                 System.out.print("Masukkan Nama Pasien :
    ");
123                 String namaPasien_3014 =
    input_3014.nextLine();
124                 System.out.print("Masukkan Keluhan     :
    ");
125                 String penyakit_3014 =
    input_3014.nextLine();
126                 daftarkanPasien_3014(
127                         namaPasien_3014,
128                         penyakit_3014);
129             } else if (pilihan_3014 == 2) {
130                 panggilPasien_3014();
131             } else if (pilihan_3014 == 3) {
132                 tampilkanAntrian_3014();
133             } else if (pilihan_3014 == 4) {
134                 System.out.print("Masukkan Nama Pasien yang
    dicari : ");
135                 String namaCari_3014 =
    input_3014.nextLine();
136                 cariPasien_3014(namaCari_3014);
137             } else if (pilihan_3014 == 5) {
138                 cekStatusAntrian_3014();
139             } else if (pilihan_3014 == 6) {
140                 System.out.println("Program selesai. Terima
    kasih!");
141             } else {
142                 System.out.println("Pilihan tidak valid!");
143             }
144         } while (pilihan_3014 != 6);
145         input_3014.close();
146     }  }
```

1. `package pekan5_2511533014;`

→ Menentukan package tempat program disimpan.

2. `import java.util.Scanner;`

→ Mengimpor class `Scanner` untuk input data dari keyboard.

3. `public class RumahSakit_2511533014 {`

→ Membuat class utama bernama `RumahSakit_2511533014`.

4. `static Pasien_2511533014 head_3014 = null;`

→ Membuat head linked list yang awalnya bernilai `null`.

5. `static int counter_3014 = 0;`

→ Membuat counter untuk nomor antrian pasien.

6. `public static void daftarkanPasien_3014(String namaPasien, String penyakit) {`

→ Method untuk menambahkan pasien ke akhir linked list.

7. `counter_3014++;`

→ Menambah nomor antrian secara otomatis.

8.`Pasien_2511533014 pasienBaru_3014 = new Pasien_2511533014(namaPasien, penyakit, counter_3014);`

→ Membuat node pasien baru.

9. `if (head_3014 == null) {`

→ Mengecek apakah linked list kosong.

10. `head_3014 = pasienBaru_3014;`

→ Jika kosong, node baru menjadi head.

11. `Pasien_2511533014 bantu_3014 = head_3014;`

→ Membuat variabel bantu untuk traversal linked list.

12. `while (bantu_3014.next_3014 != null)`

→ Menelusuri node sampai node terakhir.

13. `bantu_3014 = bantu_3014.next_3014;`

→ Memindahkan pointer ke node berikutnya.

14. `bantu_3014.next_3014 = pasienBaru_3014;`

→ Menghubungkan node terakhir dengan node baru.

15. `System.out.println("Pasien berhasil didaftarkan!");`

→ Menampilkan pesan bahwa pasien berhasil ditambahkan.

16. `System.out.println("Nomor Antrian: " + pasienBaru_3014.getNomorAntrian_3014());`

→ Menampilkan nomor antrian pasien.

17. `public static void panggilPasien_3014() {`

→ Method untuk memanggil pasien paling depan.

18. `if (head_3014 == null) {`

→ Mengecek apakah antrian kosong.

19. `System.out.println("Antrian kosong!");`

→ Menampilkan pesan jika antrian kosong.

20. `return;`

→ Menghentikan method.

21. `System.out.println("\\n=== Pasien Dipanggil ===");`

→ Menampilkan judul pasien yang dipanggil.

22. `System.out.println("Nomor Antrian : " + head_3014.getNomorAntrian_3014());`

→ Menampilkan nomor antrian pasien terdepan.

23. `System.out.println("Nama Pasien : " + head_3014.getNamaPasien_3014());`

→ Menampilkan nama pasien terdepan.

24. `System.out.println("Keluhan : " + head_3014.getPenyakit_3014());`

→ Menampilkan keluhan pasien.

25. `head_3014 = head_3014.next_3014;`

→ Menggeser head ke node berikutnya.

26. `public static void tampilkanAntrian_3014() {`

→ Method untuk menampilkan seluruh antrian pasien.

27. `System.out.println("\\n=== Daftar Antrian Pasien ===");`

→ Menampilkan judul daftar antrian.

28. `int posisi_3014 = 1;`

→ Variabel untuk nomor posisi antrian.

29. `while (bantu_3014 != null) {`

→ Menelusuri seluruh node linked list.

30. `System.out.println("Posisi Antrian : " + posisi_3014);`

→ Menampilkan posisi pasien dalam antrian.

31. `System.out.println("Nomor Antrian : " + bantu_3014.getNomorAntrian_3014());`

→ Menampilkan nomor antrian pasien.

32.System.out.println("Nama Pasien : " + bantu_3014.getNamaPasien_3014());`

→ Menampilkan nama pasien.

33. `System.out.println("Keluhan : " + bantu_3014.getPenyakit_3014());`

→ Menampilkan keluhan pasien.

34. `bantu_3014 = bantu_3014.next_3014;`

→ Berpindah ke node berikutnya.

35. `posisi_3014++;`

→ Menambah nomor posisi antrian.

36. `public static void cariPasien_3014(String namaCari_3014) {`

→ Method untuk mencari pasien berdasarkan nama.

37. `equalsIgnoreCase(namaCari_3014)`

→ Membandingkan nama tanpa membedakan huruf besar dan kecil.

38. `System.out.println("\\n=== Pasien Ditemukan ===");`

→ Menampilkan pesan jika pasien ditemukan.

39. `return;`

→ Menghentikan pencarian setelah pasien ditemukan.

40. `System.out.println("Pasien tidak ditemukan!");`

→ Menampilkan pesan jika pasien tidak ditemukan.

41. `public static void cekStatusAntrian_3014() {`

→ Method untuk mengecek status antrian.

42. `int jumlah_3014 = 0;`

→ Variabel untuk menghitung jumlah pasien.

43. `while (bantu_3014 != null) {`

→ Menelusuri seluruh linked list.

44. `jumlah_3014++;`

→ Menambah jumlah pasien.

45. `System.out.println("=== Status Antrian ===");`

→ Menampilkan judul status antrian.

46. `System.out.println("Jumlah Pasien : " + jumlah_3014);`

→ Menampilkan total pasien dalam antrian.

47. `System.out.println("Pasien Terdepan:");`

→ Menampilkan informasi pasien paling depan.

48. `System.out.println("Nomor Antrian : " + head_3014.getNomorAntrian_3014());`

→ Menampilkan nomor antrian pasien terdepan.

49. `System.out.println("Nama Pasien : " + head_3014.getNamaPasien_3014());`

→ Menampilkan nama pasien terdepan.

50. `System.out.println("Keluhan : " + head_3014.getPenyakit_3014());`

→ Menampilkan keluhan pasien terdepan.

51. `public static void main(String[] args) {`

→ Method utama program.

52. `Scanner input_3014 = new Scanner(System.in);`

→ Membuat objek Scanner untuk input keyboard.

53. `int pilihan_3014;`

→ Variabel untuk menyimpan pilihan menu.

54. `do {`

→ Perulangan menu program.

55. `System.out.println("\\n=== Antrian Rumah Sakit NIM: 2511533014 ===");`

→ Menampilkan judul menu program.

56. `pilihan_3014 = input_3014.nextInt();`

→ Membaca pilihan menu dari user.

57. `input_3014.nextLine();`

→ Membersihkan buffer Scanner.

58. `if (pilihan_3014 == 1)`

→ Memilih menu daftar pasien.

59. `String namaPasien_3014 = input_3014.nextLine();`

→ Input nama pasien.

60. `String penyakit_3014 = input_3014.nextLine();`

→ Input keluhan pasien.

61. `daftarkanPasien_3014(namaPasien_3014, penyakit_3014);`

→ Memanggil method tambah pasien.

62. `else if (pilihan_3014 == 2)`

→ Memilih menu panggil pasien.

63. `panggilPasien_3014();`

→ Memanggil method delete head.

64. `else if (pilihan_3014 == 3)`

→ Memilih menu tampilkan antrian.

65. `tampilkanAntrian_3014();`

→ Memanggil method display.

66. `else if (pilihan_3014 == 4)`

→ Memilih menu cari pasien.

67. `cariPasien_3014(namaCari_3014);`

→ Memanggil method search pasien.

68. `else if (pilihan_3014 == 5)`

→ Memilih menu cek status antrian.

69. `cekStatusAntrian_3014();`

→ Memanggil method cek status.

70. `else if (pilihan_3014 == 6)`

→ Memilih menu keluar program.

71. `System.out.println("Program selesai. Terima kasih!");`

→ Menampilkan pesan program selesai.

72. `System.out.println("Pilihan tidak valid!");`

→ Menampilkan pesan jika pilihan salah.

73. `} while (pilihan_3014 != 6);`

→ Perulangan terus berjalan sampai memilih menu 6.

74. `input_3014.close();`

→ Menutup Scanner.

75. `}`

→ Menutup class `RumahSakit_2511533014`.

### 2.6.3 Output Program

```
<terminated> RumahSakit_2511533014 [Java Application] C:\Use

=== Antrian Rumah Sakit NIM: 2511533014 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien : Budi Santoso
Masukkan Keluhan     : Demam
Pasien berhasil didaftarkan!
Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511533014 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien : Ragita
Masukkan Keluhan     : Batuk
Pasien berhasil didaftarkan!
Nomor Antrian: 2
```

```
<terminated> RumahSakit_2511533014 [Java Application] C:\Users\
=== Antrian Rumah Sakit NIM: 2511533014 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3

=== Daftar Antrian Pasien ===
Posisi Antrian : 1
Nomor Antrian  : 1
Nama Pasien    : Budi Santoso
Keluhan        : Demam
--------------------------
Posisi Antrian : 2
Nomor Antrian  : 2
Nama Pasien    : Ragita
Keluhan        : Batuk
--------------------------
```

```
<terminated> RumahSakit_2511533014 [Java Application] C:\Us
=== Antrian Rumah Sakit NIM: 2511533014 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan Nama Pasien yang dicari : Ragita

=== Pasien Ditemukan ===
Nomor Antrian : 2
Nama Pasien   : Ragita
Keluhan       : Batuk

=== Antrian Rumah Sakit NIM: 2511533014 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2
```

```
<terminated> RumahSakit_2511533014 [Java Application] C:\Us
=== Pasien Dipanggil ===
Nomor Antrian : 1
Nama Pasien   : Budi Santoso
Keluhan       : Demam

=== Antrian Rumah Sakit NIM: 2511533014 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 5
=== Status Antrian ===
Jumlah Pasien : 1
Pasien Terdepan:
Nomor Antrian : 2
Nama Pasien   : Ragita
Keluhan       : Batuk

=== Antrian Rumah Sakit NIM: 2511533014 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 6
Program selesai. Terima kasih!
```

# BAB III
# PENUTUP

## 3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa *Single Linked List* merupakan salah satu struktur data dinamis yang sangat fleksibel dalam pengelolaan data karena tidak bergantung pada ukuran memori tetap seperti array. Setiap elemen dalam *Single Linked List* direpresentasikan oleh node yang saling terhubung melalui pointer, sehingga memungkinkan proses penambahan data di berbagai posisi seperti di depan, di tengah, maupun di akhir dengan lebih efisien. Melalui implementasi program yang telah dibuat, dapat dipahami cara kerja operasi dasar seperti *insert at front*, *insert at end*, dan *insert at position*, serta proses traversal untuk menampilkan data. Dengan memahami konsep ini, mahasiswa dapat meningkatkan kemampuan dalam mengelola data secara dinamis dan lebih siap untuk mempelajari struktur data yang lebih kompleks di tahap selanjutnya.

# DAFTAR PUSTAKA

[1] M. T. Goodrich, R. Tamassia, dan M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Hoboken, NJ, USA: Wiley, 2014.

[2] Y. D. Liang, *Introduction to Java Programming and Data Structures*, 12th ed. Pearson, 2019.

[3] A. Kadir, *Data Struktur Menggunakan Java*. Yogyakarta, Indonesia: Andi Publisher, 2018.

[4] R. Munir, *Struktur Data dan Algoritma dengan Java*. Bandung, Indonesia: Informatika, 2020.

[5] Oracle, "Java Tutorials," [Online]. Available: Oracle Java Tutorials. [Accessed: May 7, 2026].

[6] GeeksforGeeks, "Linked List in Java," [Online]. Available: GeeksforGeeks Linked List in Java. [Accessed: May 7, 2026].